Input-Blind Phrase Discovery for Lossless Compression Using Hilbert–Krylov Decomposition with Infinite-Passes (HKD∞)

Michael S. Yang
Independent Researcher

Abstract

This paper investigates Hilbert–Krylov Decomposition (HKD) as a generic search framework for reversible phrase discovery prior to `bzip2` compression. Candidate phrases are discovered directly from the input, scored using the complete archive size—including dictionary, metadata, escapes, and compressed payload—and accepted only when the total archive size decreases. The resulting method is input blind in the sense that no domain vocabulary, book names, grammar rules, or file-format assumptions are embedded. Experiments reported on four large English-language corpora show aggregate savings over `bzip2 -9`, while preserving exact byte-for-byte reconstruction.

1 Introduction

Lossless data compression is fundamentally concerned with representing a source sequence using fewer bits while retaining exact recoverability. Shannon's information-theoretic framework established the central role of source entropy in determining attainable coding rates [1]. Classical coding methods such as Huffman coding provide optimal prefix codes for known symbol probabilities [2], while the Lempel–Ziv family introduced universal dictionary-based compression methods that adapt to sequential data without requiring a pre-specified source model [3, 4].

The `bzip2` compressor is based primarily on the Burrows–Wheeler block-sorting transform [5], followed by move-to-front coding, run-length coding, and entropy coding. Its mature implementation provides a useful baseline for testing whether an additional reversible preprocessing layer can improve the final archive size [6].

Hilbert–Krylov Decomposition (HKD) is used here as an input-blind search framework over reversible phrase substitutions. The method does not rely on preselected vocabulary, linguistic templates, book names, verse structure, punctuation assumptions, or any other domain-specific rule. Instead, candidate phrases are generated from repeated token sequences discovered at runtime. Each candidate phrase or connected phrase component is evaluated by constructing the complete archive and measuring its exact byte length.

The proposed approach is related to dictionary and grammar compression, including offline dictionary optimization [7], hierarchical grammar discovery [8], and universal grammar-based coding [9]. Its distinguishing feature is the use of the downstream `bzip2` archive size itself as the objective function rather than an isolated estimate of phrase-replacement savings.

2 Compression Model

Let

$$x \in \Sigma^n$$

be an input byte sequence. Let

$$D = \{p_1, \ldots, p_k\}$$

be a finite dictionary of byte phrases, and let

$$T_D(x)$$

denote the reversible transformed stream obtained by replacing non-overlapping occurrences of phrases in D with encoded dictionary references.

The complete archive cost is defined as

$$C_x(D) = L(\text{magic}) + L(\text{mode}) + L(D) + L(\text{references}) + L(\text{escapes}) + L(\text{bzip2}_9(T_D(x))),$$

where $L(\cdot)$ denotes byte length.

This objective is deliberately stricter than a phrase-level estimated gain. A phrase may save bytes before compression while disrupting Burrows–Wheeler ordering or downstream symbol statistics, thereby increasing the final archive size. Conversely, a phrase with modest direct replacement savings may improve the transformed context enough to reduce the final compressed payload by a larger amount.

Definition 1 (Input-blind phrase discovery)**.** *A phrase-discovery method is called input blind if all candidate phrases, component relations, and search parameters used to select phrases are derived from the current input and generic runtime parameters, with no domain-specific vocabulary or format rules embedded in the compressor.*

The implementations in this paper generate repeated whitespace-delimited token n-grams directly from the source bytes. Candidate phrases are then ranked only to create a bounded search pool. Final selection is always based on the exact complete archive cost $C_x(D)$.

3 Naive HKD Phrase Search

The generic HKD search begins with the empty dictionary

$$D_0 = \varnothing.$$

At iteration i, every unused candidate phrase p in the active pool is tested:

$$C_x(D_i \cup \{p\}).$$

The best strict improvement is accepted:

$$p_i^\star = \arg\min_p C_x(D_i \cup \{p\}),$$

provided that

$$C_x(D_i \cup \{p_i^\star\}) < C_x(D_i).$$

The search terminates when no one-phrase extension improves the complete archive size.

This procedure resembles greedy dictionary construction, but differs from ordinary estimated-gain heuristics because every proposed dictionary is passed through the exact reversible encoder and `bzip2 -9` backend before acceptance.

4 Connected-Component HKD Search

Single-phrase addition can miss interactions. Two phrases a and b may each fail independently,

$$C_x(D \cup \{a\}) \geq C_x(D), \qquad C_x(D \cup \{b\}) \geq C_x(D),$$

while their joint activation succeeds:

$$C_x(D \cup \{a, b\}) < C_x(D).$$

The connected solver therefore builds an interaction graph over candidate phrases. Two phrases are connected when they satisfy at least one generic relation:

1. they share one or more normalized word tokens;
2. one phrase is a byte substring of the other;
3. their occurrences occupy at least two common fixed-size byte blocks.

The solver first scores all singleton phrases, then evaluates only connected pairs and selected connected triples. This limits combinatorial growth while allowing multi-phrase components to improve upon the best single-phrase solution.

The resulting procedure is a bounded combinatorial optimization method. Its search structure is related to standard themes in combinatorial optimization, particularly neighborhood search over interacting components [10, 11].

5 Theoretical Properties

Theorem 1 (Monotone archive improvement)**.** *Let*

$$D_0, D_1, \ldots, D_t$$

be the sequence of dictionaries accepted by an HKD phrase-selection procedure. Suppose that:

1. *every transform T_{D_i} is exactly reversible;*
2. *every candidate is evaluated using the complete archive cost $C_x(D)$;*
3. *a candidate is accepted only if it strictly decreases that cost.*

Then

$$C_x(D_0) > C_x(D_1) > \cdots > C_x(D_t).$$

Proof. By the acceptance rule, the transition from D_i to D_{i+1} is performed only when

$$C_x(D_{i+1}) < C_x(D_i).$$

Applying this inequality at every accepted iteration yields

$$C_x(D_0) > C_x(D_1) > \cdots > C_x(D_t).$$

Exact reversibility ensures that every accepted archive remains a valid lossless representation of x. □

Proposition 1 (Identity fallback bound)**.** *If the identity transform is included as a candidate mode, then the selected HKD archive satisfies*

$$C_x(D^\star) \leq L(\mathrm{bzip2}_9(x)) + h,$$

where h is the fixed HKD container overhead.

Proof. The identity candidate is a valid element of the candidate set. Since the algorithm selects the smallest complete archive among the evaluated candidates, its output cannot exceed the identity archive. The identity archive consists of the raw `bzip2 -9` payload plus the fixed container overhead h. □

Theorem 2 (Connected-neighborhood exactness)**.** *Let $\mathcal{N}(D)$ be the finite set of singleton, connected-pair, and connected-triple dictionaries explicitly evaluated by the connected solver from state D. If the solver returns*

$$D^\star = \arg\min_{D' \in \mathcal{N}(D)} C_x(D'),$$

then $D^\star$ is exact with respect to the explored connected neighborhood:

$$C_x(D^\star) \leq C_x(D') \qquad \text{for all } D' \in \mathcal{N}(D).$$

Proof. The solver constructs and measures the complete archive for every member of the finite neighborhood $\mathcal{N}(D)$ and returns the member with minimum measured byte length. Therefore no evaluated connected component has lower complete archive cost. □

These results do not imply global optimality over all possible phrase dictionaries. General dictionary and grammar selection can involve a combinatorial number of interacting replacements. The theorem establishes exactness only over the explicitly evaluated connected neighborhood.

6 Phrase Compression and Input Blindness

The phrase mechanism is central to the method. A phrase is not selected because it belongs to a known language, corpus, or document family. Instead, repeated token sequences are discovered from the source itself:

$$x \longrightarrow \text{runtime tokenization} \longrightarrow \text{repeated } n\text{-grams} \longrightarrow \text{candidate phrases}.$$

The generic implementation considers token widths between two and five tokens, frequency thresholds, and phrase-length bounds. These are generic runtime parameters rather than corpus-specific knowledge. The connected implementation then derives phrase interactions from shared tokens, substring containment, and generic byte-block co-occurrence.

This distinction matters because compression improvements obtained from hard-coded text rules may disappear on unrelated inputs. Input-blind discovery instead allows different structures to emerge independently for each file. For example, the reported experiments selected different phrases or phrase components on the KJV Bible, *War and Peace*, *Moby-Dick*, and *Paradise Lost*.

Corpus	`bzip2 -9`	Naive HKD	Connected HKD	Connected savings
KJV Bible	962,764	960,728	960,728	2,036
War and Peace	888,580	888,300	888,202	378
Moby-Dick	389,166	388,929	388,887	279
Paradise Lost	145,577	145,540	145,540	37
Total	2,386,087	2,383,497	2,383,357	2,730

Table 1: Reported archive sizes in bytes.

7 Experimental Results

The reported experiments compare raw `bzip2 -9`, the naive HKD phrase solver, and the connected-component HKD solver. All reported HKD archive sizes include magic bytes, mode bytes, phrase dictionaries, references, escape handling, and the downstream `bzip2` payload.

The aggregate savings relative to `bzip2 -9` are

$$2{,}386{,}087 - 2{,}383{,}357 = 2{,}730 \text{ bytes.}$$

Measured as a percentage of the aggregate `bzip2 -9` compressed size,

$$\frac{2{,}730}{2{,}386{,}087} \times 100 \approx 0.1144\%.$$

The unweighted mean of the four per-file percentage improvements is

$$\frac{1}{4}\left(0.211474 + 0.042540 + 0.071695 + 0.025416\right) \approx 0.0878\%.$$

Thus, depending on the aggregation convention, the reported average savings are:

$$\boxed{0.1144\% \text{ of aggregate compressed size}}$$

and

$$\boxed{0.0878\% \text{ mean per-file percentage}}.$$

The connected search improves the naive HKD aggregate by

$$2{,}383{,}497 - 2{,}383{,}357 = 140 \text{ bytes.}$$

All reported archives were subjected to exact byte-for-byte reconstruction and SHA-256 verification.

8 Discussion

The reported gains are modest in percentage terms but consistent across four large literary corpora. The experimental contribution is not a claim that HKD defeats all modern lossless compressors. Rather, it shows that a generic, input-derived phrase transform can improve a mature Burrows–Wheeler compressor after every component of the reversible representation is counted.

The strongest result appears on the KJV corpus, where the automatically discovered phrase structure produces a reduction of 2,036 bytes relative to `bzip2 -9`. The connected search further improves the naive result on *War and Peace* and *Moby-Dick*.

Standard benchmark corpora remain necessary for broader evaluation. The Canterbury corpus was introduced specifically for evaluating lossless compression algorithms and is a natural next benchmark [12]. Evaluation should also include source code, markup, structured text, biological data, executables, and high-entropy inputs.

9 Limitations

The present results have several limitations:

1. only four large English-language literary corpora are reported;
2. candidate generation is bounded by a finite pool;
3. the connected search evaluates only selected singleton, pair, and triple neighborhoods;
4. no global optimum over all phrase dictionaries is proved;
5. comparison is limited primarily to `bzip2 -9`;
6. compression time is substantially greater than raw `bzip2` because many candidate archives are constructed and measured.

The method should therefore be interpreted as an experimental input-blind preprocessing strategy, not as a completed general compression breakthrough.

10 Conclusion

This paper presents an input-blind phrase-discovery framework based on Hilbert–Krylov Decomposition. The method generates phrase candidates from the runtime input, evaluates complete reversible archives, and accepts only strict archive-size improvements. A connected-component extension captures interactions between phrases that are missed by one-at-a-time addition.

Across the four reported corpora, the connected solver reduces aggregate compressed size by 2,730 bytes relative to `bzip2 -9`, corresponding to approximately 0.1144% of the aggregate compressed baseline. The connected method also improves the naive HKD result by 140 bytes in aggregate.

Future work should examine larger connected components, block-local dictionaries, exact dynamic programming over block boundaries, broader benchmark corpora, and comparisons with established grammar compressors and modern context models.

A Generic HKD Phrase Solver: Verbatim Python Module

https://github.com/yangofzeal/compress

References

[1] C. E. Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal*, vol. 27, pp. 379–423 and 623–656, 1948.

[2] D. A. Huffman, “A Method for the Construction of Minimum-Redundancy Codes,” *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.

[3] J. Ziv and A. Lempel, “A Universal Algorithm for Sequential Data Compression,” *IEEE Transactions on Information Theory*, vol. 23, no. 3, pp. 337–343, 1977.

[4] J. Ziv and A. Lempel, “Compression of Individual Sequences via Variable-Rate Coding,” *IEEE Transactions on Information Theory*, vol. 24, no. 5, pp. 530–536, 1978.

[5] M. Burrows and D. J. Wheeler, “A Block-Sorting Lossless Data Compression Algorithm,” Digital Equipment Corporation, Systems Research Center, Technical Report 124, 1994.

[6] J. Seward, *bzip2 and libbzip2*, software documentation, `https://sourceware.org/bzip2/`.

[7] N. J. Larsson and A. Moffat, “Offline Dictionary-Based Compression,” *Proceedings of the IEEE*, vol. 88, no. 11, pp. 1722–1732, 2000.

[8] C. G. Nevill-Manning and I. H. Witten, “Identifying Hierarchical Structure in Sequences: A Linear-Time Algorithm,” *Journal of Artificial Intelligence Research*, vol. 7, pp. 67–82, 1997.

[9] J. C. Kieffer and E.-H. Yang, “Grammar-Based Codes: A New Class of Universal Lossless Source Codes,” *IEEE Transactions on Information Theory*, vol. 46, no. 3, pp. 737–754, 2000.

[10] B. Korte and J. Vygen, *Combinatorial Optimization: Theory and Algorithms*, 6th ed., Springer, 2018.

[11] A. Schrijver, *Combinatorial Optimization: Polyhedra and Efficiency*, Springer, 2003.

[12] R. Arnold and T. Bell, “A Corpus for the Evaluation of Lossless Compression Algorithms,” in *Proceedings of the Data Compression Conference*, 1997.